

A dramatic mountain landscape with a sharp peak and a valley. The sky is filled with dark, heavy clouds, and mist or low clouds are swirling around the base of the mountain. The mountain's surface is rugged, with patches of brown and tan earth or rock. In the lower right, a small cluster of buildings is visible in the valley.

Lab 2 - Basic C Programming Language

String

- String จะต้องอยู่ใน "double quotes"
- โดยภาษา C จะเพิ่มอักขระพิเศษ นั่นก็คือ `\0` ต่อท้าย String นั่นก็คือ String terminator
- มีหลายชื่อเรียกเช่น
 - Null zero
 - Binary zero
 - String terminator
 - ASCII 0
 - `\0`
- ตัวโปรแกรมเมอร์เอง ไม่ต้องใส่ `\0` หรือ String terminator ลงใน "double quotes" เพราะจะถูกใส่เองโดยอัตโนมัติ
- ภาษา C ต้องใส่ `\0` ปิดท้าย เพื่อให้รู้ว่า String ที่ถูกเก็บใน memory มันจบลงที่ตรงไหน

crazy มีขนาด 6 bytes โดยมี `\0`
เป็น String terminator

c	r	a	z	y	\0
---	---	---	---	---	----

Memory

String

- ตัว `\0` จะไม่เท่ากับ `'0'` โดยถ้าดูที่ ASCII Table ตัวแรกคือ `NULL` ซึ่งมีค่าเป็น `0` แต่ `'0'` มีค่าเป็น `48`

ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[END OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

ขนาดของ String

- แล้วขนาดของ String จะมีเท่าไร?
- โดยขนาดของ String จะรวมทุกตัวอักษรและอักขระว่าง แต่ไม่รวม null zero หรือ String terminator

ตัวอย่าง

- Wednesday
- August 10
- I am here

จะเห็นได้ว่า String จะมีขนาด 9 ตัวอักษร

Character Arrays: Lists of Characters

- การประกาศ Array ใน C จะต้องมีเครื่องหมาย `[` และ `]` ต่อท้ายชื่อตัวแปร โดยต้องระบุขนาดของตัวอักษรที่จะเก็บไว้ใน `[]`

ตัวอย่าง

```
char month[10];
```

- ทำไมต้อง 10 ตัว ถึงแม้ว่าชื่อเดือนที่ยาวที่สุดคือ September นั่นก็คือ 9 ตัวอักษร แต่ก็ต้องรวม `\0` หรือ null zero ไปด้วย
- จริงๆ ควรจะเผื่อจำนวนตัวอักษรที่จะใช้ให้มากขึ้น แต่อย่าให้ขาด

```
char month[10] = "January"; // ประกาศค่าเริ่มต้น
```

month [0] [1] [2] [3] [4] [5] [6] [7] [8] [9]

J	a	n	u	a	r	y	\0	?	?
---	---	---	---	---	---	---	----	---	---

ถ้าเราเปลี่ยน month จาก January ไปเป็น March

```
1 char month[10] = "January";  
2 month[0] = 'M';  
3 month[1] = 'a';  
4 month[2] = 'r';  
5 month[3] = 'c';  
6 month[4] = 'h';
```

ถ้าเราเปลี่ยน month จาก January ไปเป็น March

1 Marchry

ถ้าเราเปลี่ยน month จาก January ไปเป็น March

```
1 char month[10] = "January";
2 month[0] = 'M';
3 month[1] = 'a';
4 month[2] = 'r';
5 month[3] = 'c';
6 month[4] = 'h';
7 month[5] = '\0'; // You must add this
```

ถ้าเราเปลี่ยน month จาก January ไปเป็น March

```
1 March
```

- ต้องเพิ่ม `\0`

การประกาศค่าเริ่มต้นให้กับ String

- อันนี้จะได้ผลลัพธ์ที่เหมือนกัน

```
char month[8] = "January";  
char month[] = "January"
```

- แต่การที่ตั้งค่าขนาดของ Array แค่ 8 ตัว แล้วในอนาคตจะใช้เก็บ String ที่ยาวขึ้นจะทำได้ ก็ควรจะต้องเผื่อไปเลย

```
char month[25] = "January";
```

- อันนี้ไม่อนุญาต จะ Error ตอนคอมไพล์

```
char month[];  
month = "April";
```

ถ้าต้องการเปลี่ยนค่าตัวแปร String จะทำอย่างไร?

- ทำแบบนี้?

```
char month[] = "January";  
month = "April";
```

ถ้าต้องการเปลี่ยนค่าตัวแปร String จะทำอย่างไร?

- ทำแบบนี้?

```
#include <stdio.h>
int main(){

    char month[] = "January";
    strcpy(month, "April");
    return 0;
}
```

ถ้าต้องการเปลี่ยนค่าตัวแปร String จะทำอย่างไร?

- ทำแบบนี้?
- คำตอบคือ `strcpy`
- แล้ว `strcpy` จะใส่ `\0` ต่อท้าย string ตัวใหม่ให้อัตโนมัติ

```
#include <stdio.h>
#include <string.h>
int main(){

    char month[] = "January";
    strcpy(month, "April");
    printf("%s", month)
    return 0;
}
```

Lab ให้พิมพ์และแก้ไข code

- ทำใน <https://onlinegdb.com>

```
/**
 * Example of strings in C Programming Language
 */

#include <stdio.h>
int main(){
    char Kid1[12];
    // Kid1 can hold an 11-character name
    // Kid2 will be 7 characters (Maddie plus null 0)
    char Kid2[] = "Maddie";
    // Kid3 is also 7 characters, but specifically defined
    char Kid3[7] = "Andrew";
    // Hero1 will be 7 characters (adding null 0!)
    char Hero1 = "Batman";
    // Hero2 will have extra room just in case
    char Hero2[34] = "Spiderman";
    char Hero3[25];
```

```
Kid1[0] = 'K'; // Kid1 is being defined
Kid1[1] = 'a'; // character-by-character
Kid1[2] = 't'; // Not efficient, but it does work
Kid1[3] = 'i';
Kid1[4] = 'e';
Kid1[5] = '\0'; // Never forget the null 0 so C
                // knows when the string ends
```

```
strcpy(Hero3, "The Incredible Hulk");
```

```
printf("%s\'s favorite hero is %s.\n", Kid1, Hero1);
printf("%s\'s favorite hero is %s.\n", Kid2, Hero2);
printf("%s\'s favorite hero is %s.\n", Kid3, Hero3);
return 0;
```

```
}
```

Preprocessor Directive

⚠ ทุกโปรแกรมต้องมี

- ใช้เรียกไฟล์ที่โปรแกรมใช้ในการทำงานร่วมกัน
- ใช้กำหนดเงื่อนไขในการคอมไพล์ให้กับโปรแกรม
- Preprocessor Directives พื้นฐานทั่วไปที่นิยมใช้
 - **#include** - ใช้สำหรับเรียกไฟล์ที่โปรแกรมใช้ในการทำงาน

- #define
- #undef
- #if
- #ifdef
- #ifndef
- #else
- #elif
- #endif
- #line
- #error
- #pragma

การใช้ #include

- วิธีการใช้งาน

```
#include <ชื่อไฟล์> // จะเรียกไฟล์ใน directory ที่กำหนดโดยตัวคอมไพเลอร์  
// OR  
#include "ชื่อไฟล์" // จะเรียกไฟล์ใน directory ที่ทำงานอยู่ในปัจจุบัน
```

- ตัวอย่าง

```
#include <stdio.h> // (เป็นการเรียกใช้ไฟล์ stdio.h เข้ามาในโปรแกรม)  
  
#include "mypro.h" // (เป็นการเรียกใช้ไฟล์ mypro.h เข้ามาในโปรแกรม)
```

ตัวอย่าง

- ทำใน <https://onlinegdb.com>

```
// addr.h
printf("\n6104 E. Oak\n");
printf("St. Paul, MN\n");
printf("        54245\n");
```

- แยกไฟล์ addr.h ออกมาโดยการกดสร้างไฟล์ใหม่ (จะมีปุ่ม  New File) ด้านบน

```
// main.c
#include <stdio.h>

int main()
{
    int age = 31;
    printf("I am %d years old", age);
    #include "addr.h"
    printf("That's my address");
    return 0;
}
```

■ OUTPUT

```
I am 31 years old  
6104 E. Oak  
St. Paul, MN  
    54245  
That's my address
```

- จุดประสงค์การสร้าง Header file เพื่อแยก code ที่เราจะใช้บ่อยๆ ออกมา แล้วไป include ในไฟล์ที่จะใช้ได้

ตัวอย่าง 2

- ทำใน <https://onlinegdb.com>

```
// letters.h
```

```
"A"  
"B"  
"C"  
"D"  
"E"  
"F"
```

- แยกไฟล์ letters.h ออกมาโดยการกดสร้างไฟล์ใหม่ (จะมีปุ่ม  New File) ด้านบน

```
// main.c  
#include <stdio.h>  
int main(){  
    char letters[] = {  
        #include "letters.h"  
    };  
    char letters2[] = {  
        "A"  
        "B"  
        "C"  
        "D"  
        "E"  
        "F"  
    };  
    char letters3[] = {"ABCDEF"};  
  
    printf("%s", letters);  
    printf("%s", letters2);  
    printf("%s", letters3);  
  
    return 0;  
}
```

■ OUTPUT

ABCDEFABCDEFABCDEF

การใช้ Constants

- `#define` เป็นการสร้างค่าคงที่ หรือ Constants ที่จะไม่มีการเปลี่ยนค่าเมื่อประกาศไปแล้ว
- Constants ไม่ถือว่าเป็นตัวแปร
- Constants จะต้องตั้งชื่อด้วยตัวพิมพ์ใหญ่

ตัวอย่าง

```
#define AGELIMIT 21
#define MYNAME "Krit Chomaitong"
#define PI 3.14159
```

- เวลาเราเขียน Code

```
if(employeeAge < AGELIMIT)
```

- Compiler จะเปลี่ยนเป็นแบบนี้ให้

```
if(employeeAge < 21)
```

ตัวอย่าง 3

- ทำใน <https://onlinegdb.com>

```
// constants.h
```

```
#define KIDS 3
#define FAMILY "The Peytons"
#define MORTGAGE_RATE 5.15
```

- แยกไฟล์ constants.h ออกมาโดยการกดสร้างไฟล์ใหม่ (จะมีปุ่ม  New File) ด้านบน

```
// main.c
```

```
#include <stdio.h>
#include <string.h>
#include "constants.h"
```

```
int main(){

    int age;
    char childname[14] = "Thomas";

    printf("\n%s have %d kids.\n", FAMILY, KIDS);
```

```
    age = 11;
    printf("The oldest, %s, is %d.\n", childname, age);

    strcpy(childname, "Christopher");
    age = 6;
    printf("The middle boy, %s, is %d.\n", childname, age);

    age = 3;
    strcpy(childname, "Benjamin");
    printf("The youngest, %s, is %d.\n", childname, age);
    return 0;
}
```

■ OUTPUT

The Peytons have 3 kids.
The oldest, Thomas, is 11.
The middle boy, Christopher, is 6.
The youngest, Benjamin, is 3.

การใช้ scanf

- `scanf` คือการรับค่าจาก keyboard
- การใช้ `scanf` จะใกล้เคียงกับ `printf` เช่นสามารถใช้ control string อย่าง `\n`, `\a`, `\t` สำหรับการแสดงข้อความออกมาทางหน้าจอ แต่ห้ามใช้ป้อนจาก keyboard เพราะทำให้ `scanf` ทำงานผิดพลาด

ตัวอย่าง

```
#include<stdio.h>
int main(){

    char firstInitial;
    char lastInitial;
    int age;
    int favorite_number;

    printf("What letter does your first name begin with?\n");
    scanf(" %c", &firstInitial);

    printf("What letter does your last name begin with?\n");
    scanf(" %c", &lastInitial);

    printf("How old are you?\n");
    scanf(" %d", &age);

    printf("What is your favorite number (integer only)?\n");
    scanf(" %d", &favorite_number);

    printf("\nYour initials are %c.%c. and you are %d years old", firstInitial, lastInitial, age);
    printf("\nYour favorite number is %d.\n\n", favorite_number);
    return 0;
}
```

คำถาม ?

1. จาก code ก่อนหน้าถ้าป้อนตัวอักษร มากกว่า 1 ตัว ให้กับ scanf ที่รับเฉพาะ %c จะเกิดอะไรขึ้น?

คำถาม ?

1. จาก code ก่อนหน้าถ้าป้อนตัวอักษร มากกว่า 1 ตัว ให้กับ scanf ที่รับเฉพาะ %c จะเกิดอะไรขึ้น?
- คำตอบคือ ทำให้ scanf ทำงานผิดพลาดแล้วมีผลต่อ scanf ในคำสั่งถัดไปด้วย

```
What letter does your first name begin with?
```

```
Kr
```

```
What letter does your last name begin with?
```

```
How old are you?
```

```
Ch
```

```
What is your favorite number (integer only)?
```

```
Your initials are K.r. and you are 1428259984 years old
```

```
Your favorite number is 32765.
```

คำถาม ?

2. ถ้าไม่ใช่ & หน้าตัวแปร ใน scanf จะเกิดอะไรขึ้น

คำถาม ?

2. ถ้าไม่ใส่ & หน้าตัวแปร ใน scanf จะเกิดอะไรขึ้น

- คำตอบคือ โปรแกรมไม่รับค่าไปเก็บไว้ในตัวแปร แต่ถ้าตัวแปรเป็น string เช่น `char fullNames[20]` จะไม่ต้องใส่ & เพราะ scanf จะรับค่าไปใส่ตัวแปร ผ่าน pointer

- ตัว scanf จะอ่าน string และจะหยุดถ้าเจออักขระว่าง นั่นก็คือถ้าป้อนมากกว่า 1 คำ จะได้แค่คำแรกคำเดียว และถ้ามี scanf ในคำสั่งถัดไปโปรแกรมก็จะทำงานผิดพลาดทันที

```
printf("What's your name\n");
scanf(" %s", fullName);

printf("What letter does your first name begin with?\n");
scanf(" %c", &firstInitial);
```

■ OUTPUT

```
What's your name
Krit Chomaitong
What letter does your first name begin with?
What letter does your last name begin with?
How old are you?
What is your favorite number (integer only)?

Your initials are C.h. and you are 0 years old
Your favorite number is 0.
```

จะเกิดอะไรขึ้นถ้าไม่ใส่ \$ ตอนโปรแกรมให้ป้อนราคา?

```
#include<stdio.h>

int main(){
    char topping[24];
    int slices;
    int month, day, year;
    float cost;

    // The first scanf will look for a floating-point variable, the cost of a pizza If the user doesn't enter
    // a $ before the cost, it could cause problems
    printf("How much does a pizza cost in your area?");
    printf("enter as $XX.XX\n");
    scanf(" $%f", &cost);

    printf("%f", cost);
    // ....
    return 0;
}
```

■ OUTPUT

```
How much does a pizza cost in your area?enter as $XX.XX
40.00
0.000000
```

ตัวแปรที่เป็น string ต้องใส่ & หรือไม่? ใน scanf

```
#include<stdio.h>

int main(){
    char topping[24];
    int slices;
    int month, day, year;
    float cost;

    // ....

    // The pizza topping is a string, so your scanf doesn't need an &
    printf("What is your favorite one-word pizza topping?\n");
    scanf(" %s", topping);

    printf("%s", topping);
    return 0;
}
```

ตัวแปรที่เป็น string ต้องใส่ & หรือไม่? ใน scanf

- คำตอบคือไม่จำเป็นต้องใส่ & หน้าตัวแปรที่เป็น string เพราะตัวแปร arrays ถือว่าเป็น pointer อยู่แล้ว

```
#include<stdio.h>

int main(){
    char topping[24];
    int slices;
    int month, day, year;
    float cost;

    // ....

    // The pizza topping is a string, so your scanf doesn't need an &
    printf("What is your favorite one-word pizza topping?\n");
    scanf(" %s", topping);

    printf("%s", topping);
    return 0;
}
```

การคำนวณ

```
#include<stdio.h>
int main(){
    float a = 19.0;
    float b = 5.0;
    float floatAnswer;

    int x = 19;
    int y = 5;
    int intAnswer;

    // Using two float variables creates an answer of 3.8
    floatAnswer = a / b;
    printf("%.1f divided by %.1f equals %.1f\n", a, b,
        floatAnswer);

    floatAnswer = x /y; // Take 2 creates an answer of 3.0
    printf("%d divided by %d equals %.1f\n", x, y,
        floatAnswer);

    // This will also be 3, as it truncates and doesn't
    // round up
    intAnswer = a / b;
```

```
    printf("%.1f divided by %.1f equals %d\n", a, b,
        intAnswer);
    return 0;
}
```

- สังเกตได้ว่า `%` จะใช้ได้เฉพาะกับข้อมูลที่เป็น integer เท่านั้น เช่น `1 % 2`

การคำนวณ

```
#include<stdio.h>
int main(){
    float a = 19.0;
    float b = 5.0;
    float floatAnswer;

    int x = 19;
    int y = 5;
    int intAnswer;

    // Using two float variables creates an answer of 3.8
    floatAnswer = a / b;
    printf("%.1f divided by %.1f equals %.1f\n", a, b,
        floatAnswer);

    floatAnswer = x /y; // Take 2 creates an answer of 3.0
    printf("%d divided by %d equals %.1f\n", x, y,
        floatAnswer);

    // This will also be 3, as it truncates and doesn't
    // round up
    intAnswer = a / b;
```

```
printf("%.1f divided by %.1f equals %d\n", a, b,
    intAnswer);
return 0;
}
```

- สังเกตได้ว่า **%** จะใช้ได้เฉพาะกับข้อมูลที่เป็น integer เท่านั้น เช่น 1 % 2

การคำนวณ

```
#include<stdio.h>
int main(){
    float a = 19.0;
    float b = 5.0;
    float floatAnswer;

    int x = 19;
    int y = 5;
    int intAnswer;

    // Using two float variables creates an answer of 3.8
    floatAnswer = a / b;
    printf("%.1f divided by %.1f equals %.1f\n", a, b,
        floatAnswer);

    floatAnswer = x /y; // Take 2 creates an answer of 3.0
    printf("%d divided by %d equals %.1f\n", x, y,
        floatAnswer);

    // This will also be 3, as it truncates and doesn't
    // round up
    intAnswer = a / b;
```

```
    printf("%.1f divided by %.1f equals %d\n", a, b,
        intAnswer);
    return 0;
}
```

- สังเกตได้ว่า **%** จะใช้ได้เฉพาะกับข้อมูลที่เป็น **integer** เท่านั้น เช่น 1 % 2

ตัวอย่างถัดไป

```
#include<stdio.h>
#define SALESTAX .07

int main(){

    // Variables for the number of tires purchased, price,
    // a before-tax total, and a total cost with tax

    int numTires;
    float tirePrice, beforeTax, netSales;

    /* Get the number of tires purchased and price per
       tire.
    */
    printf("How many tires did you purchase? ");
    scanf(" %d", &numTires);
    printf("What was the cost per tire (enter in \
        $XX.XX format)?");
    scanf(" $%f", &tirePrice);
```

```
    /* Compute the price */
    beforeTax = tirePrice * numTires;
    netSales = beforeTax + (beforeTax * SALESTAX);
    printf("You spent $%.2f on your tires\n\n\n",
        netSales);

    return 0;
}
```

■ OUTPUT

```
How many tires did you purchase? 4
What was the cost per tire (enter in $XX.XX format)?$12.33
You spent $52.77 on your tires
```

Order of Operators

- จะได้เท่าไร ?

```
int answer = 5 + 2 * 3;
```

Order of Operators

- จะได้เท่าไร ?

```
int answer = 5 + 2 * 3;
```

- จากตัวอย่างด้านบน C จะทำการคำนวณ * ก่อนแล้วมา + ดังนั้นจะได้ 11

Order of Operators

- จะได้เท่าไร ?

```
int answer = 5 + 2 * 3;
```

- จากตัวอย่างด้านบน C จะทำการคำนวณ  ก่อนแล้วมา + ดังนั้นจะได้ 11

Order of Operators

- จะได้เท่าไร ?

```
int answer = 5 + 2 * 3;
```

- จากตัวอย่างด้านบน C จะทำการคำนวณ $*$ ก่อนแล้วมา $+$ ดังนั้นจะได้ 11

Order of Operators

- จะได้เท่าไร ?

```
int answer = 5 + 2 * 3;
```

- จากตัวอย่างด้านบน C จะทำการคำนวณ $*$ ก่อนแล้วมา $+$ ดังนั้นจะได้ 11

Order of Operators

- จะได้เท่าไร ?

```
int answer = 5 + 2 * 3;
```

- จากตัวอย่างด้านบน C จะทำการคำนวณ * ก่อนแล้วมา + ดังนั้นจะได้ 11

Level	Operator	Associativity
1	() (parenthesis), [] (array element), .(structure member reference)	Left to right
2	-(negative sign), ++ (increment), -- (decrement), & (address-of), * (pointer indirection), sizeof(), ! (the not operator)	Right to left
3	* (multiplication), / (division), % (modulus)	Left to right
4	+(addition), -(subtraction)	Left to right
5	< (less than), <= (less than or equal to) > (greater than), >= (greater than or equal to)	Left to right
6	== (equal to), != (not equal to)	Left to right
7	&& (logical and)	Left to right
8	(logical or)	Left to right
9	? : (the conditional operator)	Right to left
10	=, *=, /=, %=, +=, -= (assignment operators)	Right to left
11	, (the comma operator)	Left to right

Level	Operator	Associativity
1	() (parenthesis), [] (array element), .(structure member reference)	Left to right
2	-(negative sign), ++ (increment), -- (decrement), & (address-of), * (pointer indirection), sizeof(), ! (the not operator)	Right to left
3	* (multiplication), / (division), % (modulus)	Left to right
4	+(addition), -(subtraction)	Left to right
5	< (less than), <= (less than or equal to) > (greater than), >= (greater than or equal to)	Left to right
6	== (equal to), != (not equal to)	Left to right
7	&& (logical and)	Left to right
8	(logical or)	Left to right
9	? : (the conditional operator)	Right to left
10	=, *=, /=, %=, +=, -= (assignment operators)	Right to left
11	, (the comma operator)	Left to right

■ $\text{ans} = 5 + 2 * 4 / 2 \% 3 + 10 - 3$

Level	Operator	Associativity
1	() (parenthesis), [] (array element), .(structure member reference)	Left to right
2	-(negative sign), ++ (increment), -- (decrement), & (address-of), * (pointer indirection), sizeof(), ! (the not operator)	Right to left
3	* (multiplication), / (division), % (modulus)	Left to right
4	+(addition), -(subtraction)	Left to right
5	< (less than), <= (less than or equal to) > (greater than), >= (greater than or equal to)	Left to right
6	== (equal to), != (not equal to)	Left to right
7	&& (logical and)	Left to right
8	(logical or)	Left to right
9	? : (the conditional operator)	Right to left
10	=, *=, /=, %=, +=, -= (assignment operators)	Right to left
11	, (the comma operator)	Left to right

■ $\text{ans} = 5 + \underline{2} * 4 / 2 \% 3 + 10 - 3$

Level	Operator	Associativity
1	() (parenthesis), [] (array element), .(structure member reference)	Left to right
2	-(negative sign), ++ (increment), -- (decrement), & (address-of), * (pointer indirection), sizeof(), ! (the not operator)	Right to left
3	* (multiplication), / (division), % (modulus)	Left to right
4	+(addition), -(subtraction)	Left to right
5	< (less than), <= (less than or equal to) > (greater than), >= (greater than or equal to)	Left to right
6	== (equal to), != (not equal to)	Left to right
7	&& (logical and)	Left to right
8	(logical or)	Left to right
9	? : (the conditional operator)	Right to left
10	=, *=, /=, %=, +=, -= (assignment operators)	Right to left
11	, (the comma operator)	Left to right

■ $\text{ans} = 5 + \underline{2} * \underline{4} / 2 \% 3 + 10 - 3$

Level	Operator	Associativity
1	() (parenthesis), [] (array element), .(structure member reference)	Left to right
2	-(negative sign), ++ (increment), -- (decrement), & (address-of), * (pointer indirection), sizeof(), ! (the not operator)	Right to left
3	*(multiplication), / (division), % (modulus)	Left to right
4	+(addition), -(subtraction)	Left to right
5	< (less than), <= (less than or equal to) > (greater than), >= (greater than or equal to)	Left to right
6	== (equal to), != (not equal to)	Left to right
7	&& (logical and)	Left to right
8	(logical or)	Left to right
9	? : (the conditional operator)	Right to left
10	=, *=, /=, %=, +=, -= (assignment operators)	Right to left
11	, (the comma operator)	Left to right

- ans = 5 + 2 * 4 / 2 % 3 + 10 - 3
- ans = 5 + 8 / 2 % 3 + 10 - 3

Level	Operator	Associativity
1	() (parenthesis), [] (array element), .(structure member reference)	Left to right
2	-(negative sign), ++ (increment), -- (decrement), & (address-of), * (pointer indirection), sizeof(), ! (the not operator)	Right to left
3	* (multiplication), / (division), % (modulus)	Left to right
4	+(addition), -(subtraction)	Left to right
5	< (less than), <= (less than or equal to) > (greater than), >= (greater than or equal to)	Left to right
6	== (equal to), != (not equal to)	Left to right
7	&& (logical and)	Left to right
8	(logical or)	Left to right
9	? : (the conditional operator)	Right to left
10	=, *=, /=, %=, +=, -= (assignment operators)	Right to left
11	, (the comma operator)	Left to right

- ans = 5 + 2 * 4 / 2 % 3 + 10 - 3

- ans = 5 + 8 / 2 % 3 + 10 - 3

Level	Operator	Associativity
1	() (parenthesis), [] (array element), .(structure member reference)	Left to right
2	-(negative sign), ++ (increment), -- (decrement), & (address-of), * (pointer indirection), sizeof(), ! (the not operator)	Right to left
3	* (multiplication), / (division), % (modulus)	Left to right
4	+(addition), -(subtraction)	Left to right
5	< (less than), <= (less than or equal to) > (greater than), >= (greater than or equal to)	Left to right
6	== (equal to), != (not equal to)	Left to right
7	&& (logical and)	Left to right
8	(logical or)	Left to right
9	? : (the conditional operator)	Right to left
10	=, *=, /=, %=, +=, -= (assignment operators)	Right to left
11	, (the comma operator)	Left to right

■ $\text{ans} = 5 + 2 * 4 / 2 \% 3 + 10 - 3$

■ $\text{ans} = 5 + 8 / 2 \% 3 + 10 - 3$

Level	Operator	Associativity
1	() (parenthesis), [] (array element), · (structure member reference)	Left to right
2	- (negative sign), ++ (increment), -- (decrement), & (address-of), * (pointer indirection), sizeof(), ! (the not operator)	Right to left
3	* (multiplication), / (division), % (modulus)	Left to right
4	+ (addition), - (subtraction)	Left to right
5	< (less than), <= (less than or equal to) > (greater than), >= (greater than or equal to)	Left to right
6	== (equal to), != (not equal to)	Left to right
7	&& (logical and)	Left to right
8	(logical or)	Left to right
9	? : (the conditional operator)	Right to left
10	=, *=, /=, %=, +=, -= (assignment operators)	Right to left
11	, (the comma operator)	Left to right

- $\text{ans} = 5 + 2 * 4 / 2 \% 3 + 10 - 3$
- $\text{ans} = 5 + 8 / 2 \% 3 + 10 - 3$
- $\text{ans} = 5 + 4 \% 3 + 10 - 3$

Level	Operator	Associativity
1	() (parenthesis), [] (array element), .(structure member reference)	Left to right
2	-(negative sign), ++ (increment), -- (decrement), & (address-of), * (pointer indirection), sizeof(), ! (the not operator)	Right to left
3	* (multiplication), / (division), % (modulus)	Left to right
4	+(addition), -(subtraction)	Left to right
5	< (less than), <= (less than or equal to) > (greater than), >= (greater than or equal to)	Left to right
6	== (equal to), != (not equal to)	Left to right
7	&& (logical and)	Left to right
8	(logical or)	Left to right
9	? : (the conditional operator)	Right to left
10	=, *=, /=, %=, +=, -= (assignment operators)	Right to left
11	, (the comma operator)	Left to right

- $\text{ans} = 5 + 2 * 4 / 2 \% 3 + 10 - 3$
- $\text{ans} = 5 + 8 / 2 \% 3 + 10 - 3$
- $\text{ans} = 5 + 4 \% 3 + 10 - 3$

Level	Operator	Associativity
1	() (parenthesis), [] (array element), · (structure member reference)	Left to right
2	- (negative sign), ++ (increment), -- (decrement), & (address-of), * (pointer indirection), sizeof(), ! (the not operator)	Right to left
3	* (multiplication), / (division), % (modulus)	Left to right
4	+ (addition), - (subtraction)	Left to right
5	< (less than), <= (less than or equal to) > (greater than), >= (greater than or equal to)	Left to right
6	== (equal to), != (not equal to)	Left to right
7	&& (logical and)	Left to right
8	(logical or)	Left to right
9	? : (the conditional operator)	Right to left
10	=, *=, /=, %%, +=, -= (assignment operators)	Right to left
11	, (the comma operator)	Left to right

- $\text{ans} = 5 + \underline{2} * \underline{4} / 2 \% 3 + 10 - 3$
- $\text{ans} = 5 + \underline{8} / \underline{2} \% 3 + 10 - 3$
- $\text{ans} = 5 + \underline{4} \% \underline{3} + 10 - 3$

Level	Operator	Associativity
1	() (parenthesis), [] (array element), .(structure member reference)	Left to right
2	-(negative sign), ++ (increment), -- (decrement), & (address-of), * (pointer indirection), sizeof(), ! (the not operator)	Right to left
3	* (multiplication), / (division), % (modulus)	Left to right
4	+ (addition), - (subtraction)	Left to right
5	< (less than), <= (less than or equal to) > (greater than), >= (greater than or equal to)	Left to right
6	== (equal to), != (not equal to)	Left to right
7	&& (logical and)	Left to right
8	(logical or)	Left to right
9	? : (the conditional operator)	Right to left
10	=, *=, /=, %%, +=, -= (assignment operators)	Right to left
11	, (the comma operator)	Left to right

- $\text{ans} = 5 + \underline{2} * \underline{4} / 2 \% 3 + 10 - 3$
- $\text{ans} = 5 + \underline{8} / \underline{2} \% 3 + 10 - 3$
- $\text{ans} = 5 + \underline{4} \% \underline{3} + 10 - 3$
- $\text{ans} = 5 + 1 + 10 - 3$

Level	Operator	Associativity
1	() (parenthesis), [] (array element), .(structure member reference)	Left to right
2	-(negative sign), ++ (increment), -- (decrement), & (address-of), * (pointer indirection), sizeof(), ! (the not operator)	Right to left
3	* (multiplication), / (division), % (modulus)	Left to right
4	+ (addition), - (subtraction)	Left to right
5	< (less than), <= (less than or equal to) > (greater than), >= (greater than or equal to)	Left to right
6	== (equal to), != (not equal to)	Left to right
7	&& (logical and)	Left to right
8	(logical or)	Left to right
9	? : (the conditional operator)	Right to left
10	=, *=, /=, %=, +=, -= (assignment operators)	Right to left
11	, (the comma operator)	Left to right

- $\text{ans} = 5 + 2 * 4 / 2 \% 3 + 10 - 3$
- $\text{ans} = 5 + 8 / 2 \% 3 + 10 - 3$
- $\text{ans} = 5 + 4 \% 3 + 10 - 3$
- $\text{ans} = 5 + 1 + 10 - 3$

Level	Operator	Associativity
1	() (parenthesis), [] (array element), .(structure member reference)	Left to right
2	-(negative sign), ++ (increment), -- (decrement), & (address-of), * (pointer indirection), sizeof(), ! (the not operator)	Right to left
3	* (multiplication), / (division), % (modulus)	Left to right
4	+(addition), -(subtraction)	Left to right
5	< (less than), <= (less than or equal to) > (greater than), >= (greater than or equal to)	Left to right
6	== (equal to), != (not equal to)	Left to right
7	&& (logical and)	Left to right
8	(logical or)	Left to right
9	? : (the conditional operator)	Right to left
10	=, *=, /=, %=, +=, -= (assignment operators)	Right to left
11	, (the comma operator)	Left to right

- $\text{ans} = 5 + 2 * 4 / 2 \% 3 + 10 - 3$
- $\text{ans} = 5 + 8 / 2 \% 3 + 10 - 3$
- $\text{ans} = 5 + 4 \% 3 + 10 - 3$
- $\text{ans} = 5 + 1 + 10 - 3$

Level	Operator	Associativity
1	() (parenthesis), [] (array element), .(structure member reference)	Left to right
2	-(negative sign), ++ (increment), -- (decrement), & (address-of), * (pointer indirection), sizeof(), ! (the not operator)	Right to left
3	* (multiplication), / (division), % (modulus)	Left to right
4	+ (addition), - (subtraction)	Left to right
5	< (less than), <= (less than or equal to) > (greater than), >= (greater than or equal to)	Left to right
6	== (equal to), != (not equal to)	Left to right
7	&& (logical and)	Left to right
8	(logical or)	Left to right
9	? : (the conditional operator)	Right to left
10	=, *=, /=, %=, +=, -= (assignment operators)	Right to left
11	, (the comma operator)	Left to right

- $\text{ans} = 5 + 2 * 4 / 2 \% 3 + 10 - 3$
- $\text{ans} = 5 + 8 / 2 \% 3 + 10 - 3$
- $\text{ans} = 5 + 4 \% 3 + 10 - 3$
- $\text{ans} = 5 + 1 + 10 - 3$
- $\text{ans} = 6 + 10 - 3$

Level	Operator	Associativity
1	() (parenthesis), [] (array element), · (structure member reference)	Left to right
2	- (negative sign), ++ (increment), -- (decrement), & (address-of), * (pointer indirection), sizeof(), ! (the not operator)	Right to left
3	* (multiplication), / (division), % (modulus)	Left to right
4	+ (addition), - (subtraction)	Left to right
5	< (less than), <= (less than or equal to) > (greater than), >= (greater than or equal to)	Left to right
6	== (equal to), != (not equal to)	Left to right
7	&& (logical and)	Left to right
8	(logical or)	Left to right
9	? : (the conditional operator)	Right to left
10	=, *=, /=, %%, +=, -= (assignment operators)	Right to left
11	, (the comma operator)	Left to right

- $\text{ans} = 5 + 2 * 4 / 2 \% 3 + 10 - 3$
- $\text{ans} = 5 + 8 / 2 \% 3 + 10 - 3$
- $\text{ans} = 5 + 4 \% 3 + 10 - 3$
- $\text{ans} = 5 + 1 + 10 - 3$
- $\text{ans} = 6 + 10 - 3$

Level	Operator	Associativity
1	() (parenthesis), [] (array element), · (structure member reference)	Left to right
2	- (negative sign), ++ (increment), -- (decrement), & (address-of), * (pointer indirection), sizeof(), ! (the not operator)	Right to left
3	* (multiplication), / (division), % (modulus)	Left to right
4	+ (addition), - (subtraction)	Left to right
5	< (less than), <= (less than or equal to) > (greater than), >= (greater than or equal to)	Left to right
6	== (equal to), != (not equal to)	Left to right
7	&& (logical and)	Left to right
8	(logical or)	Left to right
9	? : (the conditional operator)	Right to left
10	=, *=, /=, %=, +=, -= (assignment operators)	Right to left
11	, (the comma operator)	Left to right

- $\text{ans} = 5 + \underline{2} * \underline{4} / 2 \% 3 + 10 - 3$
- $\text{ans} = 5 + \underline{8} / \underline{2} \% 3 + 10 - 3$
- $\text{ans} = 5 + \underline{4} \% \underline{3} + 10 - 3$
- $\text{ans} = \underline{5} + \underline{1} + 10 - 3$
- $\text{ans} = \underline{6} + \underline{10} - 3$

Level	Operator	Associativity
1	() (parenthesis), [] (array element), · (structure member reference)	Left to right
2	- (negative sign), ++ (increment), -- (decrement), & (address-of), * (pointer indirection), sizeof(), ! (the not operator)	Right to left
3	* (multiplication), / (division), % (modulus)	Left to right
4	+ (addition), - (subtraction)	Left to right
5	< (less than), <= (less than or equal to) > (greater than), >= (greater than or equal to)	Left to right
6	== (equal to), != (not equal to)	Left to right
7	&& (logical and)	Left to right
8	(logical or)	Left to right
9	? : (the conditional operator)	Right to left
10	=, *=, /=, %=, +=, -= (assignment operators)	Right to left
11	, (the comma operator)	Left to right

- $\text{ans} = 5 + 2 * 4 / 2 \% 3 + 10 - 3$
- $\text{ans} = 5 + 8 / 2 \% 3 + 10 - 3$
- $\text{ans} = 5 + 4 \% 3 + 10 - 3$
- $\text{ans} = 5 + 1 + 10 - 3$
- $\text{ans} = 6 + 10 - 3$
- $\text{ans} = 16 - 3$

Level	Operator	Associativity
1	() (parenthesis), [] (array element), · (structure member reference)	Left to right
2	- (negative sign), ++ (increment), -- (decrement), & (address-of), * (pointer indirection), sizeof(), ! (the not operator)	Right to left
3	* (multiplication), / (division), % (modulus)	Left to right
4	+ (addition), - (subtraction)	Left to right
5	< (less than), <= (less than or equal to) > (greater than), >= (greater than or equal to)	Left to right
6	== (equal to), != (not equal to)	Left to right
7	&& (logical and)	Left to right
8	(logical or)	Left to right
9	? : (the conditional operator)	Right to left
10	=, *=, /=, %=, +=, -= (assignment operators)	Right to left
11	, (the comma operator)	Left to right

- $\text{ans} = 5 + \underline{2} * \underline{4} / 2 \% 3 + 10 - 3$
- $\text{ans} = 5 + \underline{8} / \underline{2} \% 3 + 10 - 3$
- $\text{ans} = 5 + \underline{4} \% \underline{3} + 10 - 3$
- $\text{ans} = \underline{5} + \underline{1} + 10 - 3$
- $\text{ans} = \underline{6} + \underline{10} - 3$
- $\text{ans} = \underline{16} - 3$

Level	Operator	Associativity
1	() (parenthesis), [] (array element), · (structure member reference)	Left to right
2	- (negative sign), ++ (increment), -- (decrement), & (address-of), * (pointer indirection), sizeof(), ! (the not operator)	Right to left
3	* (multiplication), / (division), % (modulus)	Left to right
4	+ (addition), - (subtraction)	Left to right
5	< (less than), <= (less than or equal to) > (greater than), >= (greater than or equal to)	Left to right
6	== (equal to), != (not equal to)	Left to right
7	&& (logical and)	Left to right
8	(logical or)	Left to right
9	? : (the conditional operator)	Right to left
10	=, *=, /=, %%, +=, -= (assignment operators)	Right to left
11	, (the comma operator)	Left to right

- $\text{ans} = 5 + 2 * 4 / 2 \% 3 + 10 - 3$
- $\text{ans} = 5 + 8 / 2 \% 3 + 10 - 3$
- $\text{ans} = 5 + 4 \% 3 + 10 - 3$
- $\text{ans} = 5 + 1 + 10 - 3$
- $\text{ans} = 6 + 10 - 3$
- $\text{ans} = 16 - 3$

Level	Operator	Associativity
1	() (parenthesis), [] (array element), · (structure member reference)	Left to right
2	- (negative sign), ++ (increment), -- (decrement), & (address-of), * (pointer indirection), sizeof(), ! (the not operator)	Right to left
3	* (multiplication), / (division), % (modulus)	Left to right
4	+ (addition), - (subtraction)	Left to right
5	< (less than), <= (less than or equal to) > (greater than), >= (greater than or equal to)	Left to right
6	== (equal to), != (not equal to)	Left to right
7	&& (logical and)	Left to right
8	(logical or)	Left to right
9	? : (the conditional operator)	Right to left
10	=, *=, /=, %%, +=, -= (assignment operators)	Right to left
11	, (the comma operator)	Left to right

- ans = 5 + 2 * 4 / 2 % 3 + 10 - 3
- ans = 5 + 8 / 2 % 3 + 10 - 3
- ans = 5 + 4 % 3 + 10 - 3
- ans = 5 + 1 + 10 - 3
- ans = 6 + 10 - 3
- ans = 16 - 3
- ans = 13

Level	Operator	Associativity
1	() (parenthesis), [] (array element), . (structure member reference)	Left to right
2	- (negative sign), ++ (increment), -- (decrement), & (address-of), * (pointer indirection), sizeof(), ! (the not operator)	Right to left
3	* (multiplication), / (division), % (modulus)	Left to right
4	+ (addition), - (subtraction)	Left to right
5	< (less than), <= (less than or equal to), > (greater than), >= (greater than or equal to)	Left to right
6	== (equal to), != (not equal to)	Left to right
7	&& (logical and)	Left to right
8	(logical or)	Left to right
9	? : (the conditional operator)	Right to left
10	=, *=, /=, %=, +=, -= (assignment operators)	Right to left
11	, (the comma operator)	Left to right

- $\text{ans} = 5 + \underline{2} * \underline{4} / 2 \% 3 + 10 - 3$
- $\text{ans} = 5 + \underline{8} / \underline{2} \% 3 + 10 - 3$
- $\text{ans} = 5 + \underline{4} \% \underline{3} + 10 - 3$
- $\text{ans} = \underline{5} + \underline{1} + 10 - 3$
- $\text{ans} = \underline{6} + \underline{10} - 3$
- $\text{ans} = \underline{16} - \underline{3}$
- $\text{ans} = \underline{13}$

Level	Operator	Associativity
1	() (parenthesis), [] (array element), .(structure member reference)	Left to right
2	-(negative sign), ++ (increment), -- (decrement), & (address-of), * (pointer indirection), sizeof(), ! (the not operator)	Right to left
3	*(multiplication), / (division), % (modulus)	Left to right
4	+(addition), -(subtraction)	Left to right
5	< (less than), <= (less than or equal to) > (greater than), >= (greater than or equal to)	Left to right
6	== (equal to), != (not equal to)	Left to right
7	&& (logical and)	Left to right
8	(logical or)	Left to right
9	? : (the conditional operator)	Right to left
10	=, *=, /=, %=, +=, -= (assignment operators)	Right to left
11	, (the comma operator)	Left to right

- จะแก้ปัญหาคำนวณว่าจะให้คำนวณก่อนหลังได้อย่างไร?

- จะแก้ปัญหาคำนวณว่าจะให้คำนวณก่อนหลังได้อย่างไร?
- ใช้ (กับ)

- จะแก้ปัญหาคำนวณว่าจะให้คำนวณก่อนหลังได้อย่างไร?
- ใช้ (กับ)

- จะแก้ปัญหาคำนวณว่าจะให้คำนวณก่อนหลังได้อย่างไร?
- ใช้ (กับ)

- จะแก้ปัญหาคำนวณว่าจะให้คำนวณก่อนหลังได้อย่างไร?
- ใช้ (กับ)
- ตัวอย่าง $\text{ans} = (5 + 2) * 3 = 21$

- จะแก้ปัญหาคำนวณว่าจะให้คำนวณก่อนหลังได้อย่างไร?
- ใช้ (กับ)
- ตัวอย่าง $\text{ans} = \underline{(5 + 2) * 3} = 21$

- จะแก้ปัญหาคำนวณว่าจะให้คำนวณก่อนหลังได้อย่างไร?
- ใช้ (กับ)
- ตัวอย่าง $\text{ans} = \underline{(5 + 2)} * 3 = 21$

- จะแก้ปัญหาคำนวณว่าจะให้คำนวณก่อนหลังได้อย่างไร?
- ใช้ (กับ)
- ตัวอย่าง $\text{ans} = \underline{(5 + 2)} * \underline{3} = \underline{21}$

- จะแก้ปัญหาคำนวณว่าจะให้คำนวณก่อนหลังได้อย่างไร?
- ใช้ (กับ)
- ตัวอย่าง $\text{ans} = (5 + 2) * 3 = 21$
- ตัวอย่าง $\text{avg} = (i + j + k + l) / 4$

- จะแก้ปัญหาคำนวณว่าจะให้คำนวณก่อนหลังได้อย่างไร?
- ใช้ (กับ)
- ตัวอย่าง $\text{ans} = (5 + 2) * 3 = 21$
- ตัวอย่าง $\text{avg} = (i + j + k + l) / 4$

- จะแก้ปัญหาคำนวณว่าจะให้คำนวณก่อนหลังได้อย่างไร?
- ใช้ (กับ)
- ตัวอย่าง $\text{ans} = (5 + 2) * 3 = 21$
- ตัวอย่าง $\text{avg} = (i + j + k + l) / 4$

- ถ้ามีตัวแปรประเภท integer ตั้งแต่ a ถึง j โดยที่ทุกตัวมีค่าเริ่มต้นเป็น 9 จะประกาศยังไง?

```
int a,b,c,d,e,f,g,h,i,j;
```

- แล้วถ้า $a = 5 * (b = 2)$ จะได้เท่าไร?

- ตัวอย่างด้านล่างได้เท่าไร?

```
int newValue, oldValue = 10, factor = 1;  
newValue = oldValue - -factor;  
printf("%d", newValue);
```

- ถ้ามีตัวแปรประเภท integer ตั้งแต่ a ถึง j โดยที่ทุกตัวมีค่าเริ่มต้นเป็น 9 จะประกาศยังไง?

```
int a,b,c,d,e,f,g,h,i,j;  
a=9;b=9;c=9;d=9;e=9;f=9;g=9;h=9;i=9;j=9;
```

- แล้วถ้า $a = 5 * (b = 2)$ จะได้เท่าไร?
- ตัวอย่างด้านล่างได้เท่าไร?

```
int newValue, oldValue = 10, factor = 1;  
newValue = oldValue - -factor;  
printf("%d", newValue);
```

- ถ้ามีตัวแปรประเภท integer ตั้งแต่ a ถึง j โดยที่ทุกตัวมีค่าเริ่มต้นเป็น 9 จะประกาศอย่างไร?

```
int a,b,c,d,e,f,g,h,i,j;  
a = b = c = d = e = f = g = h = i = j = 9;
```

- แล้วถ้า $a = 5 * (b = 2)$ จะได้เท่าไร?
- ตัวอย่างด้านล่างได้เท่าไร?

```
int newValue, oldValue = 10, factor = 1;  
newValue = oldValue - -factor;  
printf("%d", newValue);
```

- ถ้ามีตัวแปรประเภท integer ตั้งแต่ a ถึง j โดยที่ทุกตัวมีค่าเริ่มต้นเป็น 9 จะประกาศยังไง?

```
int a,b,c,d,e,f,g,h,i,j;  
a = b = c = d = e = f = g = h = i = j = 9;
```

- แล้วถ้า $a = 5 * (b = 2)$ จะได้เท่าไร? 10

- ตัวอย่างด้านล่างได้เท่าไร?

```
int newValue, oldValue = 10, factor = 1;  
newValue = oldValue - -factor;  
printf("%d", newValue);
```

- ถ้ามีตัวแปรประเภท integer ตั้งแต่ a ถึง j โดยที่ทุกตัวมีค่าเริ่มต้นเป็น 9 จะประกาศอย่างไร?

```
int a,b,c,d,e,f,g,h,i,j;  
a = b = c = d = e = f = g = h = i = j = 9;
```

- แล้วถ้า $a = 5 * (b = 2)$ จะได้เท่าไร? 10
- ตัวอย่างด้านล่างได้เท่าไร? 11

```
int newValue, oldValue = 10, factor = 1;  
newValue = oldValue - -factor;  
printf("%d", newValue);
```

ตัวอย่าง

```
#include<stdio.h>
int main(){
    int grade1, grade2, grade3, grade4;
    float averageGrade, gradeDelta, percentDiff;

    /* The student got 88s on the first and third test,
       so a multiple assignment statement works. */
    grade1 = grade3 = 88;

    grade2 = 79;

    // The user needs to enter the fourth grade
    printf("What did you get on the fourth test");
    printf(" (An integer between 0 and 100)?");
    scanf(" %d", &grade4);
```

```
    averageGrade = (grade1+grade2+grade3+grade4)/4;
    printf("Your average is %.1f.\n", averageGrade);

    gradeDelta = 95 - averageGrade;
    percentDiff = 100 * ((95 - averageGrade) / 95);
    printf("Your grade is %.1f points lower than the ",
           gradeDelta);
    printf("top grade in the class (95)\n");
    printf("You are %.1f percent behind ", percentDiff);
    printf("that grade!\n\n\n");
    return 0;
}
```

■ OUTPUT

```
What did you get on the fourth test
(An integer between 0 and 100)?50
Your average is 76.0.
Your grade is 19.0 points lower than the
top grade in the class (95)
You are 20.0 percent behind that grade!
```

Compound Assignment

- เป็นตัวดำเนินการใช้สำหรับปรับปรุงหรืออัปเดตค่าของตัวแปร โดยมีข้อกำหนดว่าข้อมูลใหม่นั้นต้องมีความสัมพันธ์กับข้อมูลเดิม

```
int lossCount = 1;  
  
lossCount = lossCount + 1; /* Adds 1 to lossCount variable */
```

ตัวอย่าง

```
#include<stdio.h>
int main(){

    int ctr = 0;
    ctr = ctr + 1; // increases counter to 1
    printf("Counter is at %d.\n", ctr);
    ctr = ctr + 1; // increases counter to 2
    printf("Counter is at %d.\n", ctr);
    ctr = ctr + 1; // increases counter to 3
    printf("Counter is at %d.\n", ctr);
    ctr = ctr + 1; // increases counter to 4
    printf("Counter is at %d.\n", ctr);
    ctr = ctr + 1; // increases counter to 5
    printf("Counter is at %d.\n", ctr);
    ctr = ctr - 1; // decreases counter to 4
    printf("Counter is at %d.\n", ctr);
    ctr = ctr - 1; // decreases counter to 3
    printf("Counter is at %d.\n", ctr);
    ctr = ctr - 1; // decreases counter to 2
    printf("Counter is at %d.\n", ctr);
    ctr = ctr - 1; // decreases counter to 1
    printf("Counter is at %d.\n", ctr);
    return 0;
}
```

■ OUTPUT

```
Counter is at 1.
Counter is at 2.
Counter is at 3.
Counter is at 4.
Counter is at 5.
Counter is at 4.
Counter is at 3.
Counter is at 2.
Counter is at 1.
```

- ถ้าต้องการเขียนให้สั้นลง เราสามารถใช้ compound operator อย่าง += ได้

```
int lossCount = 0;  
lossCount = lossCount + 1;
```

- ถ้าเป็น *=

```
int sales = 10;  
sales = sales * 1.25;
```

- ถ้าต้องการเขียนให้สั้นลง เราสามารถใช้ compound operator อย่าง += ได้

```
int lossCount = 0;  
lossCount += 1;
```

- ถ้าเป็น *=

```
int sales = 10;  
sales = sales * 1.25;
```

- ถ้าต้องการเขียนให้สั้นลง เราสามารถใช้ compound operator อย่าง `+=` ได้

```
int lossCount = 0;  
lossCount += 1;
```

- ถ้าเป็น `*=`

```
int sales = 10;  
sales *= 1.25;
```

- ถ้าต้องการเขียนให้สั้นลง เราสามารถใช้ compound operator อย่าง += ได้

```
int lossCount = 0;  
lossCount += 1;
```

- ถ้าเป็น *=

```
int sales = 10;  
sales *= 1.25;
```

- ตัวอย่าง Compound Operator

Compound Operator	Example	Equivalent Statement
*=	total *= 1.25;	total = total * 1.25;
/=	amt /= factor;	amt = amt / factor;
%=	days %= 3;	days = days % 3;
+=	count += 1;	count = count + 1;
-=	quantity -= 5;	quantity = quantity - 5;

ตัวอย่าง

```
#include<stdio.h>
int main(){
    int ctr = 0;
    ctr += 1; // increases counter to 1
    printf("Counter is at %d.\n", ctr);
    ctr += 1; // increases counter to 2
    printf("Counter is at %d.\n", ctr);

    printf("Counter is at %d.\n", ctr += 1);
    ctr += 1; // increases counter to 4
    printf("Counter is at %d.\n", ctr);

    printf("Counter is at %d.\n", ctr += 1);
    ctr -= 1; // decreases counter to 4
    printf("Counter is at %d.\n", ctr);
    printf("Counter is at %d.\n", ctr -= 1);
    printf("Counter is at %d.\n", ctr -= 1);
    printf("Counter is at %d.\n", ctr -= 1);
}
```

■ OUTPUT

```
Counter is at 1.
Counter is at 2.
Counter is at 3.
Counter is at 4.
Counter is at 5.
Counter is at 4.
Counter is at 3.
Counter is at 2.
Counter is at 1.
```

- ถ้าไปเจอแบบนี้จะตอบอะไร?

```
int total = 5;  
total *= 2 + 3;
```

- ถ้าไปเจอแบบนี้จะตอบอะไร?

```
int total = 5;  
total *= 2 + 3;
```

- 13 เพราะคิดว่าจะต้องทำ * ก่อนแล้วค่อยมา + แต่จริงๆ แล้วไม่ใช่

- ถ้าไปเจอแบบนี้จะตอบอะไร?

```
int total = 5;  
total *= 2 + 3;
```

- 13 เพราะคิดว่าจะต้องทำ * ก่อนแล้วค่อยมา + แต่จริงๆ แล้วไม่ใช่

- ถ้าไปเจอแบบนี้จะตอบอะไร?

```
int total = 5;  
total *= 2 + 3;
```

- 13 เพราะคิดว่าจะต้องทำ * ก่อนแล้วค่อยมา + แต่จริงๆ แล้วไม่ใช่

- ถ้าไปเจอแบบนี้จะตอบอะไร?

```
int total = 5;  
total *= 2 + 3;
```

- 13 เพราะคิดว่าจะต้องทำ * ก่อนแล้วค่อยมา + แต่จริงๆ แล้วไม่ใช่

- ถ้าไปเจอแบบนี้จะตอบอะไร?

```
int total = 5;  
total *= 2 + 3;
```

- 13 เพราะคิดว่าจะต้องทำ * ก่อนแล้วค่อยมา + แต่จริงๆ แล้วไม่ใช่

```
int total = 5;  
total = total * (2+3);
```

- ถ้าไปเจอแบบนี้จะตอบอะไร?

```
int total = 5;  
total *= 2 + 3;
```

- 13 เพราะคิดว่าจะต้องทำ * ก่อนแล้วค่อยมา + แต่จริงๆ แล้วไม่ใช่

```
int total = 5;  
total = total * (2+3);
```

- 25 เพราะว่า *= อยู่ต่ำกว่า + ถ้าดูจากตาราง

- ถ้าไปเจอแบบนี้จะตอบอะไร?

```
int total = 5;  
total *= 2 + 3;
```

- 13 เพราะคิดว่าจะต้องทำ * ก่อนแล้วค่อยมา + แต่จริงๆ แล้วไม่ใช่

```
int total = 5;  
total = total * (2+3);
```

- 25 เพราะว่า *= อยู่ต่ำกว่า + ถ้าดูจากตาราง

- ถ้าไปเจอแบบนี้จะตอบอะไร?

```
int total = 5;  
total *= 2 + 3;
```

- 13 เพราะคิดว่าจะต้องทำ * ก่อนแล้วค่อยมา + แต่จริงๆ แล้วไม่ใช่

```
int total = 5;  
total = total * (2+3);
```

- 25 เพราะว่า *= อยู่ต่ำกว่า + ถ้าดูจากตาราง

■ ถ้าไปเจอแบบนี้จะตอบอะไร?

```
int total = 5;  
total *= 2 + 3;
```

- 13 เพราะคิดว่าจะต้องทำ * ก่อนแล้วค่อยมา + แต่จริงๆ แล้วไม่ใช่

```
int total = 5;  
total = total * (2+3);
```

- 25 เพราะว่า *= อยู่ต่ำกว่า + ถ้าดูจากตาราง

Level	Operator	Associativity
1	() (parenthesis), [] (array element), .(structure member reference)	Left to right
2	- (negative sign), ++ (increment), -- (decrement), & (address-of), * (pointer indirection), sizeof(), ! (the not operator)	Right to left
3	* (multiplication), / (division), % (modulus)	Left to right
4	+ (addition), - (subtraction)	Left to right
5	< (less than), <= (less than or equal to) > (greater than), >= (greater than or equal to)	Left to right
6	== (equal to), != (not equal to)	Left to right
7	&& (logical and)	Left to right
8	(logical or)	Left to right
9	? : (the conditional operator)	Right to left
10	=, *=, /=, %=, +=, -= (assignment operators)	Right to left
11	, (the comma operator)	Left to right

การ cast ค่า

- จะเป็นการแปลงชนิดของข้อมูลไปเป็นอีกชนิดหนึ่ง ซึ่งเป็นการแปลงแบบชั่วคราว
- ตัวอย่าง

```
int age = 6;

printf("%f\n", (float)age);

int number = 15;
int yrsService = 5;
float value;
value = (float)(number - 10 * yrsService);

printf("%f", value);
```

■ OUTPUT

```
6.000000
-35.000000
```